



Pianificazione Appelli Progettazione di Applicazioni Web Full Stack

Gabriele Fiorucci 740746

Scaletta

1. Diagramma ER
2. Diagramma degli utenti
3. Matrice CRUD
4. BackEnd
5. Esempio End to End
6. Frontend
7. Progetto Coarse
8. Progettazione dell'ipertesto

Diagramma ER

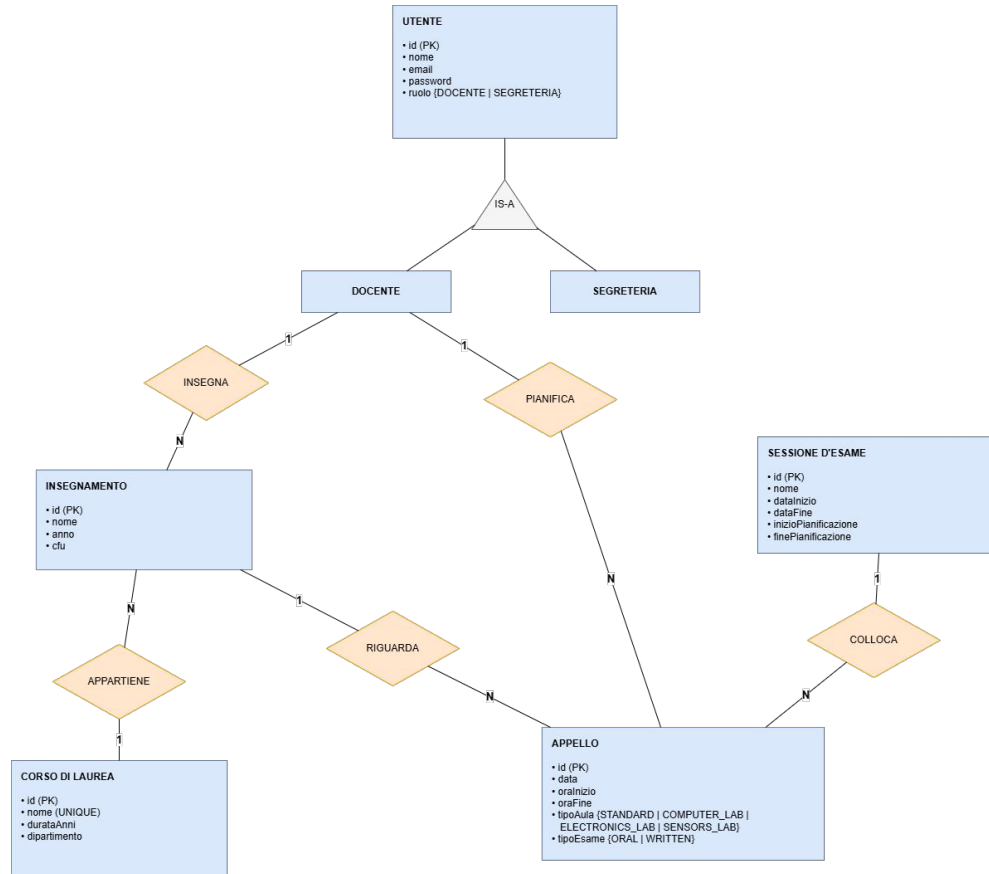
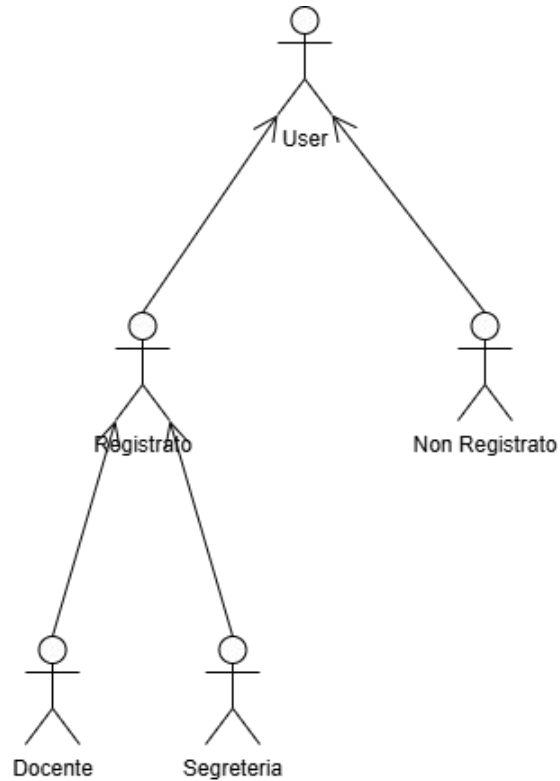


Diagramma Degli Utenti



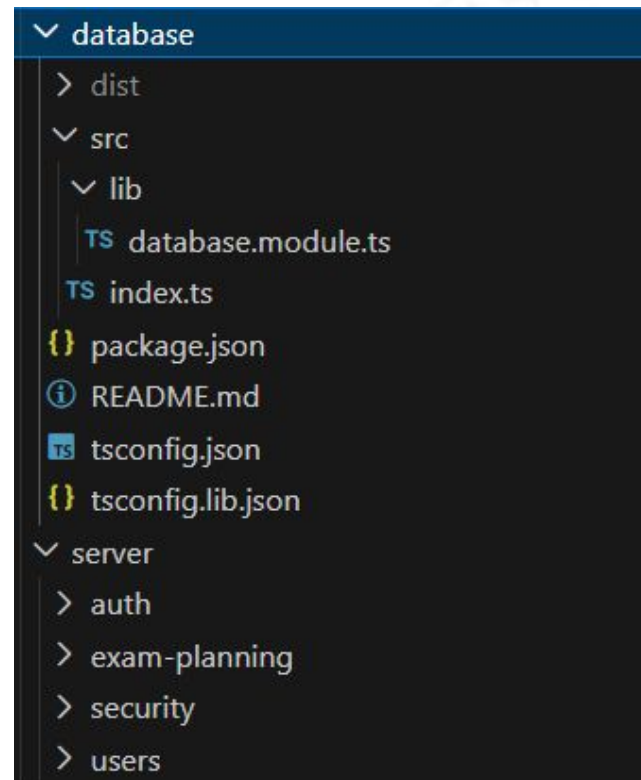
Matrice CRUD

	DOCENTE	SEGRETERIA	NON REGISTRATO
Docente	(RUD) (Sé stesso)	CRUD	---
Segreteria	---	CR(UD) (Sé stesso)	---
Corso di Laurea	(R) (I propri)	CRUD	----
Appello	(CUD)R (I propri)	RUD	---
Sessione	R	CRUD	---
Materia	(R) (Le proprie)	CRUD	---

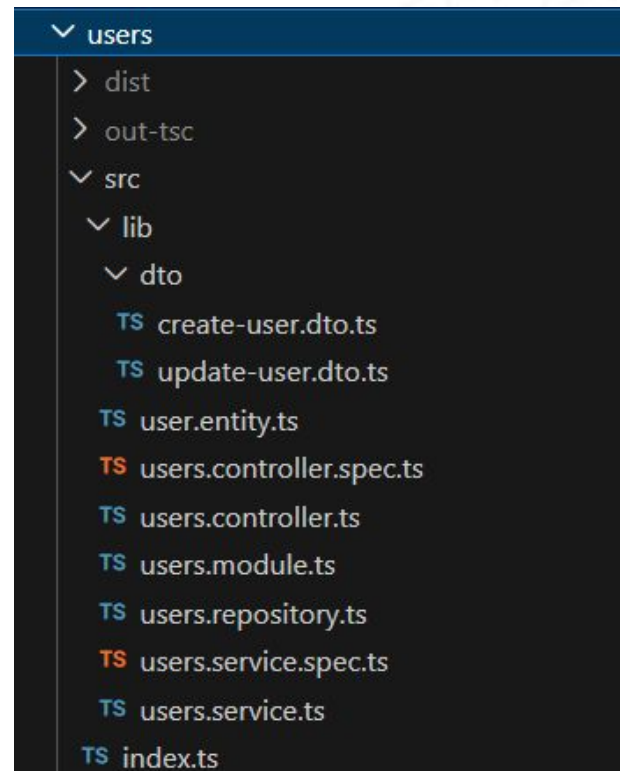
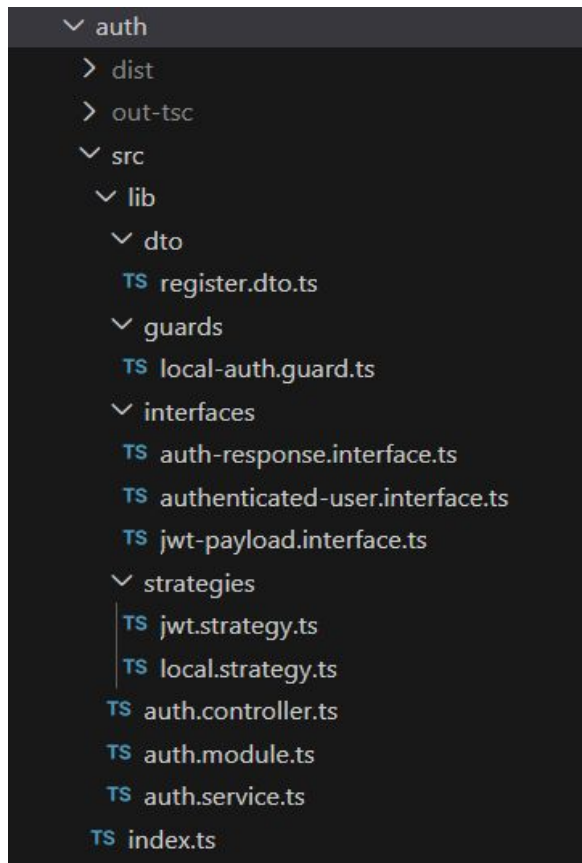
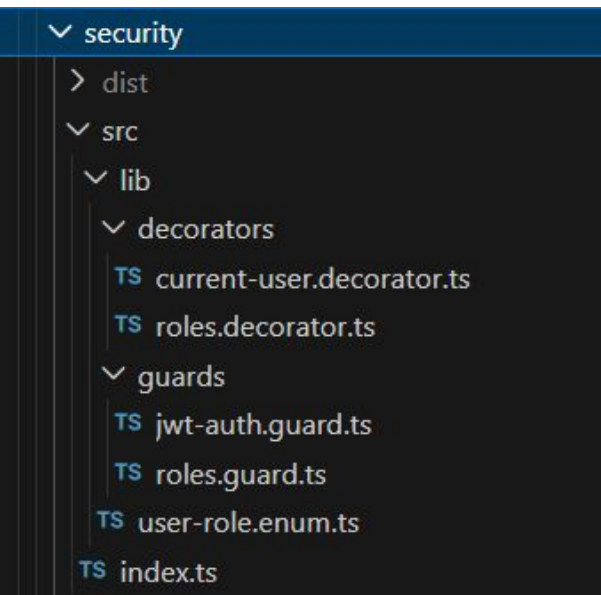


BackEnd

- Cartella **database** utilizzata per la connessione al database PostGres tramite [database.module.ts](#)
- Il backend è contenuto in /libs/server e si divide in diverse sottocartelle specializzate
- /auth per autenticazione
- /security per la gestione dei permessi post autenticazione
- /users per la gestione degli utenti
- /exam-planning contiene la vera e propria logica di backend del progetto



Authentication, Security & Users



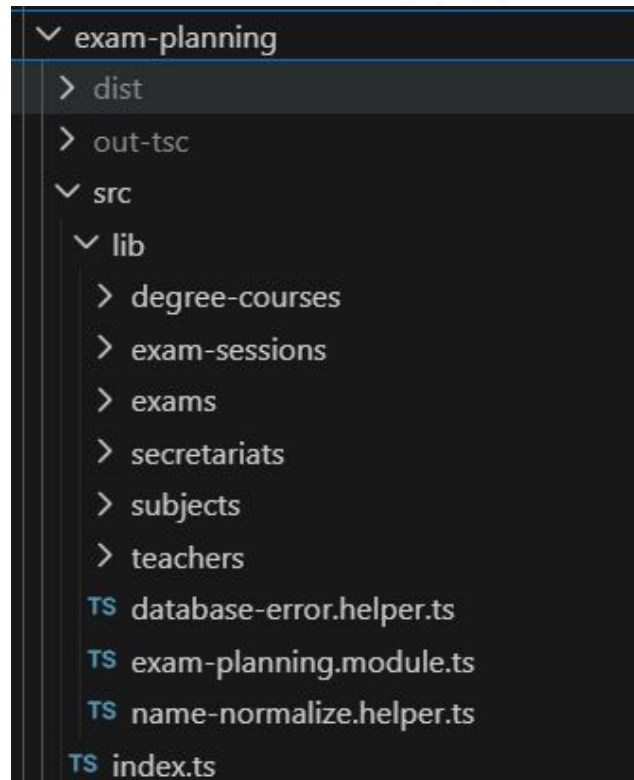
Database Module

- La connessione al database è effettuata tramite database module con **TypeOrmModule**.
- Un unico modulo centralizzato per la configurazione del database
- Variabili d'ambiente lette direttamente dal file .env per garantire univocità
- Le entità vengono caricate automaticamente tramite **autoLoadEntities**, senza doverle dichiarare centralmente

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'postgres',
      host: requireEnv('PG_HOST'),
      port: Number(requireEnv('PG_PORT')),
      username: requireEnv('PG_USERNAME'),
      password: requireEnv('PG_PASSWORD'),
      database: requireEnv('PG_DATABASE'),
      autoLoadEntities: true,
      synchronize: true,
    }),
  ],
  exports: [TypeOrmModule],
})
export class DatabaseModule {}
```

Logica BackEnd

- La logica vera e propria del progetto risiede in `/exam-planning/src/lib/`
- Per ogni entità del progetto è stata definita una cartella contenente **DTO**, **interfacce**, **entity**, **repository**, **controller** e **service**.
- Tutte le entità seguono lo stesso pattern
- Vi sono anche helper per gestione degli errori e parsing dei nomi per evitare di avere nomi duplicati, anche in casi speciali come nomi che differiscono per spazi o maiuscole



Un esempio di flusso end to end

Entity

```
@Entity('exams')
@Unique(['date', 'subject'])
export class ExamEntity {

  @PrimaryGeneratedColumn({ name: 'id' })
  examId: number;

  @Column({ type: 'date', nullable: false })
  date: Date;

  @Column({ type: 'int', nullable: false })
  startHour: number;

  @Column({ type: 'int', nullable: false })
  endHour: number;

  @Column({ type: 'enum', enum: RoomType, nullable: false })
  roomType: RoomType;

  @Column({ type: 'enum', enum: ExamType })
  type: ExamType;

  @ManyToOne(() => SubjectEntity, (subject) => subject.exams, {
    nullable: false,
    eager: true,
    onDelete: 'RESTRICT',
  })
  subject: SubjectEntity;

  @ManyToOne(() => ExamSessionEntity, (session) => session.exams, {
    nullable: false,
    eager: true,
    onDelete: 'RESTRICT',
  })
  examSession: ExamSessionEntity;

  @ManyToOne(() => TeacherEntity, (teacher) => teacher.exams, {
    nullable: false,
    eager: true,
    onDelete: 'RESTRICT',
  })
  teacher: TeacherEntity;
```

+

DTO

```
export class CreateExamDto {

  @ApiModelProperty({ example: '2026-06-15', description: "Data dell'appello (ISO YYYY-MM-DD)" })
  @IsDateString({}, { message: "La data dell'esame deve essere una data valida (formato AAAA-MM-DD)" })
  date: string;

  @ApiModelProperty({ example: 9, description: "Ora di inizio dell'appello (intero, 0-24)" })
  @IsInt({ message: "L'ora di inizio esame deve essere un numero intero" })
  @Min(0, { message: "L'ora di inizio esame non può essere minore di 0" })
  @Max(24, { message: "L'ora di inizio esame non può essere maggiore di 24" })
  startHour: number;

  @ApiModelProperty({ example: 11, description: "Ora di fine dell'appello (intero, 0-24, deve essere maggiore di startHour)" })
  @IsInt({ message: "L'ora di fine esame deve essere un numero intero" })
  @Min(0, { message: "L'ora di fine esame non può essere minore di 0" })
  @Max(24, { message: "L'ora di fine esame non può essere maggiore di 24" })
  endHour: number;

  @ApiModelProperty({ enum: RoomType, example: RoomType.STANDARD, description: "Tipo di aula richiesto" })
  @IsEnum(RoomType, { message: "Il tipo di aula richiesto non è valido" })
  roomType: RoomType;

  @ApiModelProperty({ enum: ExamType, example: ExamType.ORAL })
  @IsEnum(ExamType, { message: "Il tipo di esame non è valido" })
  type: ExamType;

  @ApiModelProperty({ example: 1, description: 'ID della materia' })
  @IsInt({ message: "L'identificativo della materia deve essere un numero intero" })
  @IsPositive({ message: "L'identificativo della materia deve essere un numero positivo" })
  subjectId: number;

  @ApiModelProperty({ example: 1, description: "ID della sessione d'esame" })
  @IsInt({ message: "L'identificativo della sessione d'esame deve essere un numero intero" })
  @IsPositive({ message: "L'identificativo della sessione d'esame deve essere un numero positivo" })
  examSessionId: number;
}
```

Repository

```
@Injectable()
export class ExamsRepository {
  constructor(
    @InjectRepository(ExamEntity)
    private readonly repository: Repository<ExamEntity>,
  ) {}

  /** Tutti gli appelli, ordinati per id. */
  findAll(): Promise<ExamEntity[]> {
    return this.repository.find({ order: { examId: 'ASC' } });
  }

  /** Un appello per id; `null` se non esiste. */
  findById(examId: number): Promise<ExamEntity | null> {
    return this.repository.findOne({ where: { examId } });
  }

  /** Gli appelli di un docente, ordinati per data. */
  findByTeacher(teacherId: number): Promise<ExamEntity[]> {
    return this.repository.find({
      where: { teacher: { teacherId } },
      order: { date: 'ASC' },
    });
  }
}
```

Service

```
@Injectable()
export class ExamsService {
  constructor(
    private readonly repository: ExamsRepository,
    private readonly subjectsRepository: SubjectsRepository,
    private readonly examSessionsRepository: ExamSessionsRepository,
    private readonly teachersRepository: TeachersRepository,
  ) {}

  // FK annidate {id, name}; date come stringa ISO (YYYY-MM-DD).
  private toItem(exam: ExamEntity): ExamListItem {
    return {
      id: exam.examId,
      date: this.toIso(exam.date),
      startHour: exam.startHour,
      endHour: exam.endHour,
      roomType: exam.roomType,
      type: exam.type,
      subject: {
        id: exam.subject.subjectId,
        name: exam.subject.name,
      },
      examSession: {
        id: exam.examSession.examSessionId,
        name: exam.examSession.name,
      },
      teacher: {
        id: exam.teacher.teacherId,
        name: exam.teacher.userName,
      },
    };
  }

  /** Recupera l'ENTITÀ esame per id (uso interno); 404 se non esiste. */
  private async getEntityById(examId: number): Promise<ExamEntity> {
    const exam = await this.repository.findById(examId);
    if (!exam) {
      throw new NotFoundException(`Esame con id ${examId} non trovato`);
    }
    return exam;
  }
}
```

Controller

ExamPages (frontend)

```
@ApiTags('Exams APIs')
@Controller('exams')
export class ExamsController {
  constructor(private readonly service: ExamsService) {}

  @Get()
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.SEGRETERIA)
  @ApiBearerAuth()
  findAll() {
    return this.service.findAll();
  }

  @Get('/mine')
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.DOCENTE)
  @ApiBearerAuth()
  findOwn(@CurrentUser() currentUser: AuthenticatedUser) {
    return this.service.findOwnExams(currentUser);
  }

  @Get('/by-teacher/:teacherId')
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.SEGRETERIA)
  @ApiBearerAuth()
  findByTeacher(@Param('teacherId', ParseIntPipe) teacherId: number) {
    return this.service.findByTeacher(teacherId);
  }

  @Get('/by-course-year')
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.DOCENTE)
  @ApiBearerAuth()
  findByCourseAndYear(
    @Query('degreeCourseId', ParseIntPipe) degreeCourseId: number,
    @Query('year', ParseIntPipe) year: number,
  ) {
    return this.service.findByCourseAndYear(degreeCourseId, year);
  }
}
```

```
export function ExamsPage() {
  const navigate = useNavigate();
  const user = getCurrentUser();
  const isSegreteria = user?.role === 'SEGRETERIA';

  const [items, setItems] = useState<ExamListItem[]>([]);
  const [sessions, setSessions] = useState<ExamSessionListItem[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // Carica esami (lista giusta in base al ruolo) e sessioni (per le date/ordine).
  useEffect(() => {
    const loadExams = isSegreteria ? fetchExams : fetchMyExams;
    Promise.all([loadExams(), fetchExamSessions()])
      .then(([exams, allSessions]) => {
        setItems(exams);
        setSessions(allSessions);
      })
      .catch((err) => setError(err instanceof Error ? err.message : 'Errore'))
      .finally(() => setLoading(false));
  }, [isSegreteria]);

  async function handleDelete(exam: ExamListItem) {
    if (
      !window.confirm(
        `Eliminare l'esame di "${exam.subject.name}" del ${formatDate(exam.date)}?`
      )
    ) {
      return;
    }
    try {
      await deleteExam(exam.id);
      // Aggiornamento ottimistico: tolgo la riga senza rifare la fetch.
      setItems((prev) => prev.filter((e) => e.id !== exam.id));
    } catch (err) {
      alert(err instanceof Error ? err.message : "Errore durante l'eliminazione");
    }
  }
}
```

CreateExamPage

```
export function CreateExamPage() {
  const navigate = useNavigate();
  // Valori iniziali opzionali passati dal calendario (click su giorno verde):
  // /exams/new?subjectId=&date=&examSessionId=. Se assenti, form vuoto.
  const [searchParams] = useSearchParams();

  const [subjectId, setSubjectId] = useState(searchParams.get('subjectId') ?? '');
  const [examSessionId, setExamSessionId] = useState(
    searchParams.get('examSessionId') ?? ''
  );

  const [date, setDate] = useState(searchParams.get('date') ?? '');
  const [startHour, setStartHour] = useState(9);
  const [endHour, setEndHour] = useState(11);
  const [type, setType] = useState<ExamType>('ORAL' as ExamType);
  const [roomType, setRoomType] = useState<RoomType>('STANDARD' as RoomType);

  const [subjects, setSubjects] = useState<SubjectListItem[]>([]);
  const [sessions, setSessions] = useState<ExamSessionListItem[]>([]);
  const [error, setError] = useState<string | null>(null);
  const [saving, setSaving] = useState(false);

  // Carica le opzioni dei due <select> al montaggio.
  useEffect(() => {
    fetchMySubjects()
      .then(setSubjects)
      .catch(() => setSubjects([]));
    // Mostra solo le sessioni in cui ha ancora senso pianificare: quelle la cui
    // finestra di pianificazione non è già chiusa (planningEndDate >= oggi).
    // Tiene sia le sessioni pianificabili ora sia quelle future (finestra non
    // ancora aperta) ed esclude le passate/chiusure, in linea col vincolo backend.
    fetchExamSessions()
      .then((all) => {
        const today = todayIso();
        setSessions(all.filter((s) => s.planningEndDate >= today));
      })
      .catch(() => setSessions([]));
  }, []);
```

Descrizione del flusso End To End

- Una richiesta dal FrontEnd passa dal controller, poi al service, repository fino al database.
- Al ritorno:
 1. Entity + DTO definiscono la struttura dei dati e la validazione
 2. Repository comunica con il database
 3. Service per la logica di dominio e autorizzazione
 4. Controller espone le API REST
 5. Nel Frontend xPages crea la lista di dati e CreatexPage crea la pagina vera e propria

FrontEnd

- Il FrontEnd si compone di 3 macro aree: Login, Docenti e Segreteria
- La pagina di Login è una pagina default comune a tutti gli utenti non registrati
- La sezione dedicata ai Docenti si apre automaticamente nel momento in cui un docente si logga, allo stesso modo la sezione dedicata alla Segreteria
- Sezione Docenti e sezione Segreteria sono per lo più speculari, con una differenza di permessi e funzionalità.

Login

Pianificazione Appelli

Accedi con le tue credenziali

Email

nome.cognome@unibs.it

Password

Inserisci la password

Accedi

Dashboard Docenti

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) Mario Rossi DOCENTE [Logout](#)

Dashboard

1
Sessioni aperte alla pianificazione

4
I tuoi appelli

4
Le tue materie

Scegli un insegnamento per vedere i giorni in cui puoi inserire un appello

— Vista generale (sessioni e pianificazione) —

Oggi Indietro Avanti

giugno 2026

lun	mar	mer	gio	ven	sab	dom
01	02	03	04	05	06	07
08	09	10	11	12	13	14
15	16	17	18	19	20	21

Esami (Docenti)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#)

Mario Rossi [DOCENTE](#) [Logout](#)

Esami

I tuoi appelli

[+ Nuovo esame](#)

Sessione Autunnale 2026

Data	Orario	Materia	Tipo	Aula	Azioni
25/09/2026	09:00–11:00	Scienza delle Costruzioni	Scritto	Aula standard	Modifica Elimina

Luglio 2026

01/07/2026	09:00–11:00	Ingegneria Del Software A	Orale	Aula standard	Modifica Elimina
01/07/2026	09:00–11:00	Analisi Matematica	Orale	Aula standard	Modifica Elimina

Sessione Invernale 2025

02/01/2026	14:00–16:00	Scienza delle Costruzioni	Orale	Aula standard	<i>Esame concluso</i>
------------	-------------	---------------------------	-------	---------------	-----------------------

Aggiunta di un nuovo esame

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) Mario Rossi DOCENTE [Logout](#)

Nuovo esame

Materia
Seleziona una materia... ▼

Sessione
Seleziona una sessione... ▼

Data
gg / mm / aaaa 

Ora inizio
9 ▼

Ora fine
11 ▼

Tipo
Orale ▼

Aula
Aula standard ▼

[Annulla](#) [Crea esame](#)

Sessioni (docente)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) Mario Rossi DOCENTE [Logout](#)

Sessioni d'esame

In corso

Sessioni attualmente aperte

Nome	Sessione	Pianificazione appelli
Giugno 2026	01/06/2026 – 29/06/2026	01/05/2026 – 15/05/2026

Pianificate

Sessioni future

Nome	Sessione	Pianificazione appelli
Luglio 2026	01/07/2026 – 30/07/2026	15/06/2026 – 30/06/2026
Settembre 2026	01/09/2026 – 30/09/2026	01/07/2026 – 31/07/2026
Sessione Autunnale 2026	19/09/2026 – 09/10/2026	20/08/2026 – 09/09/2026

Passate

Sessioni concluse

Nome	Sessione	Pianificazione appelli
Sessione Invernale 2025	23/12/2025 – 12/01/2026	13/11/2025 – 18/12/2025

Insegnamenti (docente)

Appelli Esami Sessioni Insegnamenti Corsi di laurea						Mario Rossi	DOCENTE	Logout
Insegnamenti								
I tuoi insegnamenti								
Nome	Anno	CFU	Insegnante	Corso di laurea	Dipartimento			
Analisi Matematica	1	12	Mario Rossi	Ingegneria Civile	Dipartimento di Ingegneria Civile			
Ingegneria Del Software A	1	6	Mario Rossi	Ingegneria Informatica	Dipartimento di Ingegneria Dell'Informazione (DII)			
Ingegneria Del Software B	1	6	Mario Rossi	Ingegneria Informatica	Dipartimento di Ingegneria Dell'Informazione (DII)			
Scienza delle Costruzioni	2	9	Mario Rossi	Ingegneria Civile	Dipartimento di Ingegneria Civile			

Corsi di laurea (docente)

[Appelli](#)[Esami](#)[Sessioni](#)[Insegnamenti](#)[Corsi di laurea](#)[Mario Rossi](#) [DOCENTE](#)[Logout](#)

Corsi di laurea

I corsi a te assegnati

Nome	Durata (anni)	Dipartimento
Ingegneria Civile	3	Dipartimento di Ingegneria Civile
Ingegneria Informatica	3	Dipartimento di Ingegneria Dell'Informazione (DII)

Area personale docente

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) Mario Rossi DOCENTE [Logout](#)

Area personale

Nome

Email (non modificabile)

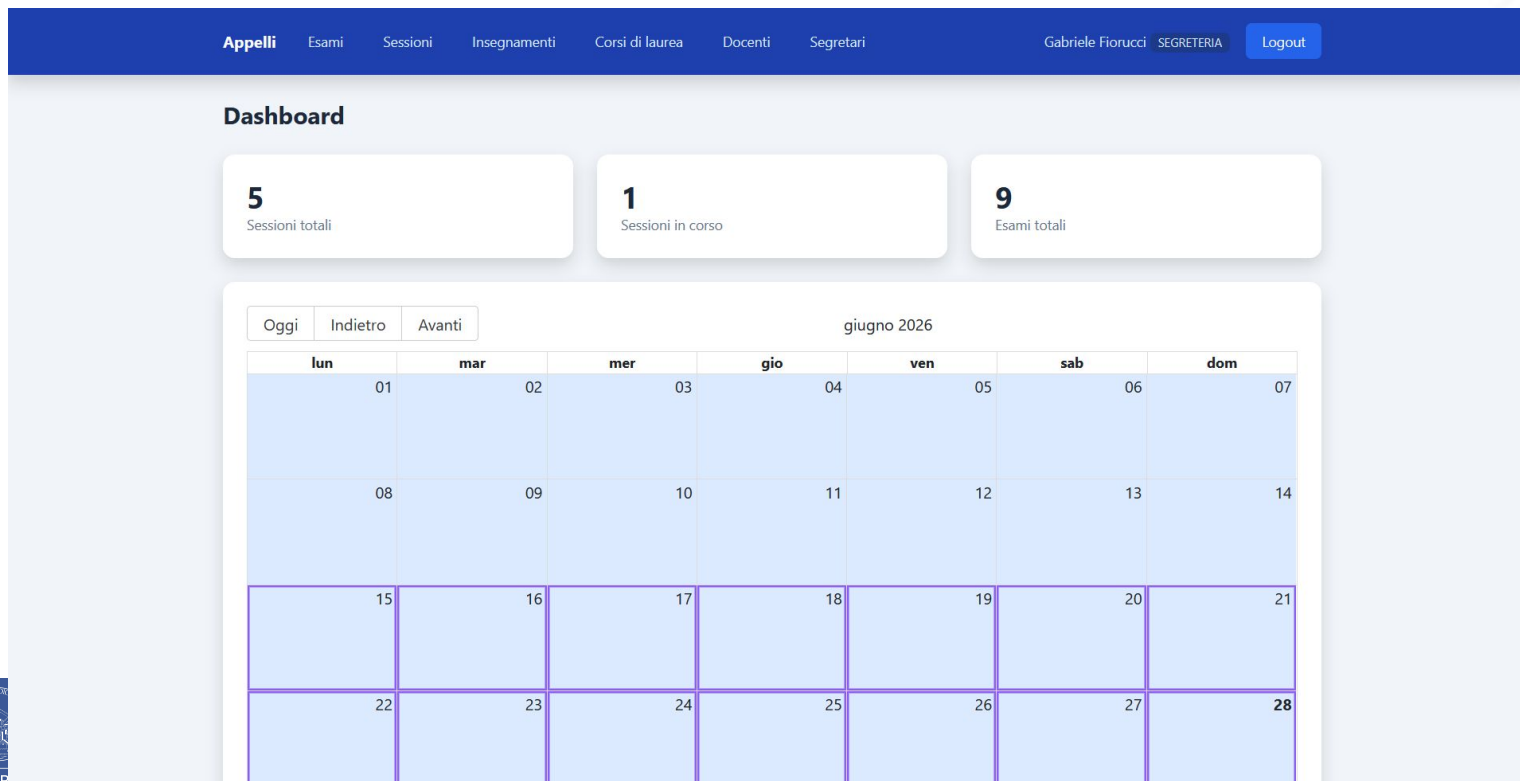
 

Nuova password (opzionale)

Almeno 8 caratteri, una lettera maiuscola e un simbolo tra ? ^ ! # @

[Elimina account](#) [Indietro](#) [Salva modifiche](#)

Dashboard Segreteria



Esami (segreteria)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci [SEGRETARIA](#) [Logout](#)

Esami

Sessione Autunnale 2026

Data	Orario	Materia	Docente	Tipo	Aula	Azioni
25/09/2026	09:00–11:00	Scienza delle Costruzioni	Mario Rossi	Scritto	Aula standard	Modifica Elimina
28/09/2026	14:00–17:00	Programmazione Web	Devis Bianchini	Orale	Laboratorio informatico	Modifica Elimina

Luglio 2026

Data	Orario	Materia	Docente	Tipo	Aula	Azioni
01/07/2026	09:00–11:00	Basi di Dati	Devis Bianchini	Orale	Aula standard	Modifica Elimina
01/07/2026	09:00–11:00	Programmazione Web	Devis Bianchini	Orale	Aula standard	Modifica Elimina
01/07/2026	09:00–11:00	Analisi Matematica	Mario Rossi	Orale	Aula standard	Modifica Elimina
01/07/2026	09:00–11:00	Ingegneria Del Software A	Mario Rossi	Orale	Aula standard	Modifica Elimina
02/07/2026	09:00–11:00	Analisi Matematica	Devis Bianchini	Orale	Aula standard	Modifica Elimina

Sessione Invernale 2025

Sessioni (segreteria)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci SEGRETERIA [Logout](#)

Sessioni d'esame

[+ Nuova sessione](#)

In corso
Sessioni attualmente aperte

Nome	Sessione	Pianificazione appelli	Azioni
Giugno 2026	01/06/2026 – 29/06/2026	01/05/2026 – 15/05/2026	Modifica Elimina

Pianificate
Sessioni future

Nome	Sessione	Pianificazione appelli	Azioni
Luglio 2026	01/07/2026 – 30/07/2026	15/06/2026 – 30/06/2026	Modifica Elimina
Settembre 2026	01/09/2026 – 30/09/2026	01/07/2026 – 31/07/2026	Modifica Elimina
Sessione Autunnale 2026	19/09/2026 – 09/10/2026	20/08/2026 – 09/09/2026	Modifica Elimina

Passate
Sessioni concluse

Nome	Sessione	Pianificazione appelli	Azioni
Sessione Invernale 2025	23/12/2025 – 12/01/2026	13/11/2025 – 18/12/2025	Modifica Elimina

Nuova sessione (segreteria)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci [SEGRETARIA](#) [Logout](#)

Nuova sessione d'esame

Nome

Es. Giugno 2026

Periodo della sessione

Inizio sessione

gg / mm / aaaa

Fine sessione

gg / mm / aaaa

Finestra di pianificazione appelli

Inizio pianificazione

gg / mm / aaaa

Fine pianificazione

gg / mm / aaaa

Annulla

Crea sessione

Insegnamenti (segreteria)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci [SEGRETARIA](#) [Logout](#)

Insegnamenti

[+ Nuovo insegnamento](#)

Nome	Anno	CFU	Insegnante	Corso di laurea	Dipartimento	Azioni	
Analisi Matematica	1	12	Devis Bianchini	Ingegneria Informatica	Dipartimento di Ingegneria Dell'Informazione (DII)	Modifica	Elimina
Analisi Matematica	1	12	Mario Rossi	Ingegneria Civile	Dipartimento di Ingegneria Civile	Modifica	Elimina
Ingegneria Del Software A	1	6	Mario Rossi	Ingegneria Informatica	Dipartimento di Ingegneria Dell'Informazione (DII)	Modifica	Elimina
Ingegneria Del Software B	1	6	Mario Rossi	Ingegneria Informatica	Dipartimento di Ingegneria Dell'Informazione (DII)	Modifica	Elimina
Basi di Dati	2	9	Devis Bianchini	Ingegneria Informatica	Dipartimento di Ingegneria Dell'Informazione (DII)	Modifica	Elimina
Scienza delle Costruzioni	2	9	Mario Rossi	Ingegneria Civile	Dipartimento di Ingegneria Civile	Modifica	Elimina
Programmazione Web	3	6	Devis Bianchini	Ingegneria Informatica	Dipartimento di Ingegneria Dell'Informazione (DII)	Modifica	Elimina

Nuovo Insegnamento (segreteria)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci [SEGRETERIA](#) [Logout](#)

Nuovo insegnamento

Nome
Es. Analisi Matematica

Corso di laurea
Seleziona un corso...

Docente titolare
Seleziona un docente...

Anno
1

CFU
6

[Annulla](#) [Crea insegnamento](#)

Corsi di laurea (segreteria)

[Appelli](#)[Esami](#)[Sessioni](#)[Insegnamenti](#)[Corsi di laurea](#)[Docenti](#)[Segretari](#)[Gabriele Fiorucci](#) [SEGRETARIA](#)[Logout](#)

Corsi di laurea

[+ Nuovo corso](#)

Nome	Durata (anni)	Dipartimento	Azioni	
Ingegneria Civile	3	Dipartimento di Ingegneria Civile	Modifica	Elimina
Ingegneria Informatica	3	Dipartimento di Ingegneria Dell'Informazione (DII)	Modifica	Elimina

Nuovo corso di laurea (segreteria)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci [SEGRETARIA](#) [Logout](#)

Nuovo corso di laurea

Nome

Durata (anni)

Dipartimento

[Annulla](#) [Crea corso](#)

Docenti (segreteria)

[Appelli](#)[Esami](#)[Sessioni](#)[Insegnamenti](#)[Corsi di laurea](#)[Docenti](#)[Segretari](#)[Gabriele Fiorucci](#) [SEGRETERIA](#)[Logout](#)

Docenti

[+ Nuovo docente](#)

Nome	Email	Azioni	
Devis Bianchini	devis.bianchini@unibs.it	Modifica	Elimina
Mario Rossi	mario.rossi@unibs.it	Modifica	Elimina

Nuovo docente (segreteria)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci [SEGRETERIA](#) [Logout](#)

Nuovo docente

Nome

Es. Mario Rossi

Email

Es. mario.rossi@unibs.it



Password

Almeno 8 caratteri, una lettera maiuscola e un simbolo tra ? ^ ! # @

Annulla

Crea docente

Segretari (segreteria)

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci [SEGRETERIA](#) [Logout](#)

Segretari

[+ Nuovo segretario](#)

Nome	Email	Azioni
Gabriele Fiorucci Tu	gabriele.fiorucci@unibs.it	Modifica Elimina
Anna Bianchi	anna.bianchi@unibs.it	—

Nuovo Segretario

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) [Gabriele Fiorucci](#) [SEGRETERIA](#) [Logout](#)

Nuovo segretario

Nome

Es. Anna Bianchi

Email

Es. anna.bianchi@unibs.it

Password

Almeno 8 caratteri, una lettera maiuscola e un simbolo tra ? ^ ! # @

Annulla

Crea segretario

Modifica Segretario / Area Personale

[Appelli](#) [Esami](#) [Sessioni](#) [Insegnamenti](#) [Corsi di laurea](#) [Docenti](#) [Segretari](#) Gabriele Fiorucci [SEGRETARIA](#) [Logout](#)

Area personale

Nome

Email (non modificabile)

Nuova password (opzionale)

Almeno 8 caratteri, una lettera maiuscola e un simbolo tra ? ^ ! # @

[Elimina account](#) [Indietro](#) [Salva modifiche](#)

Regole di dominio

- Segreteria gestisce tutte le risorse, Docente solo i propri dati.
- Il Docente può creare esami solo per le proprie materie.
- Il Docente può modificare o eliminare solo i propri esami e solo se non sono già passati.
- La Segreteria può modificare o eliminare qualsiasi esame senza restrizioni.
- Gli esami devono essere pianificati all'interno dell'intervallo di date della sessione di riferimento.

- Il Docente può pianificare esami solo durante la finestra di pianificazione della sessione.
- Non è consentita la pianificazione di esami nei giorni di sabato e domenica.
- Non sono ammessi conflitti: è possibile avere un solo esame per corso di laurea, anno e giorno.
- Non sono ammessi duplicati: una materia può avere un solo esame per sessione.
- L'ora di inizio dell'esame deve essere precedente all'ora di fine.
- La creazione di Docenti e Segretari avviene in modo atomico insieme al relativo utente (rollback in caso di errore).

- La Segreteria può modificare o eliminare esclusivamente il proprio profilo (self-only).
- L'eliminazione di un Docente è consentita solo se non ha materie o esami associati.

Progetto Coarse

